

Design of Parallel Systems

1 Overview

Beyond query processing and data storage, a large-scale parallel database must address the operational challenges of managing massive volumes of data. This includes Parallel Loading, High Availability, and Online Maintenance.

2 Availability and Resilience

In a system with hundreds or thousands of nodes, the probability of hardware failure (disk or processor) increases significantly.

The Scalability of Failure

If a single processor fails once every 5 years, a parallel system with 100 processors will experience a failure approximately every ****18 days****. A poorly designed system that stops functioning upon any component failure becomes inherently unreliable at scale.

2.1 Fault Tolerance Strategies

Modern systems (e.g., Teradata, IBM Informix XPS) ensure availability through:

- **Data Replication:** Storing copies of data across at least two different processors.
- **Dynamic Routing:** Automatically directing data requests to backup sites when a primary node fails.
- **Skew-Averse Mirroring:** Instead of replicating all data from Processor A to a single Processor B (which would double B's load during A's failure), replicas are ****partitioned across multiple processors**** to balance the emergency load.

3 Online Reorganization

For databases in the terabyte/petabyte range, operations like reindexing or schema changes can take days. It is unacceptable for the system to be offline during this period.

Online Operations

Online reindexing and ****Online schema changes**** allow updates (Insert/Delete/Update) to continue while the maintenance task is running.

Mechanism: Online Index Construction

Unlike traditional reindexing which locks the table, the online process:

1. Starts building the index on a static snapshot of the data.
2. Maintains a **delta log** or side-table to track updates that occur during construction.
3. Merges the logged changes into the new index before making it active.

4 Modern Parallel Database Architectures

Many contemporary parallel databases follow a "cluster of instances" model, leveraging existing relational engines.

4.1 The Multi-Node Instance Model

Products like **Netezza**, **Greenplum**, and **Aster Data** run an instance of an underlying database on every node in a cluster.

- **PostgreSQL Base:** Netezza, Greenplum, and Aster Data use PostgreSQL as their core engine.
- **Benefits:** Developers can focus on high-level concerns (partitioning, parallel query optimization, inter-processor communication) while the base engine handles local storage and transaction management.
- **Cost Efficiency:** Using open-source bases like PostgreSQL significantly reduces the per-node software licensing cost.

4.2 Database Appliances

Some vendors (Netezza, DATAlegro) sell integrated **Data Warehouse Appliances**.

Appliances

These are pre-configured bundles of hardware (compute/storage) and optimized software, allowing organizations to deploy massive parallel databases with minimal internal engineering effort.

Reliability and maintenance are as critical as raw performance in the design of production-grade parallel database systems.